
MiniGames App

TP note – Seance 1 sur 3

Developpement Android avec Jetpack Compose

Theme de la seance :

Kotlin & Jetpack Compose

Premiers pas avec Compose, gestion d'etat locale, timer interactif

Formation : CNAM – 1ere annee

Travail : En binome

Duree : 4h

Auteur : Jordan Hiertz

Ce sujet est le premier volet d'un TP progressif sur 3 seances.
L'application sera enrichie a chaque seance.

Vue d'ensemble

Dans ce TP, vous allez développer une application Android de mini-jeux appelée **MiniGames App**. Ce projet sera votre fil conducteur sur les 3 séances : chaque séance introduit de nouveaux concepts et enrichit l'application existante.

Progression sur 3 séances

Séance 1	Kotlin + Compose	Ecran d'accueil, jeu de reaction
Séance 2	Architecture + Navigation	ViewModel, NavHost, jeu de mots
Séance 3	Base de données	Room, scores, leaderboard

A la fin de cette première séance, vous disposerez d'une application fonctionnelle composée d'un écran d'accueil et d'un premier mini-jeu.

Objectifs pédagogiques

1. Découvrir la syntaxe Kotlin.
2. Créer des interfaces déclaratives avec **Jetpack Compose**.
3. Gérer l'état local d'un composable avec `remember` et `mutableStateOf`.
4. Utiliser les **coroutines** pour implémenter un timer réactif.
5. Comprendre le mécanisme de **recomposition** : l'UI se met à jour automatiquement lorsque l'état change.

1. Mise en route

1.1. Création du projet

1. Ouvrez **Android Studio** et créez un nouveau projet : *File > New > New Project*.
2. Choisissez le template **Empty Activity** (Pas Empty Views Activity).
3. Configurez le projet comme suit :
 - **Name** : MiniGamesApp ou celui de votre choix
 - **Package name** : com.nom1nom2.minigames
 - **Language** : Kotlin
 - **Minimum SDK** : API 26 (Android 8.0)
4. Lancez une première compilation pour vérifier que le projet démarre correctement.

1.2. Dependances

Les dépendances Jetpack Compose sont incluses par défaut avec le template **Empty Activity**. Vous devez en revanche ajouter manuellement le support des **coroutines**. Les dépendances se déclarent dans deux fichiers : `gradle/libs.versions.toml` d'abord, puis `build.gradle.kts` (module).

Dans `gradle/libs.versions.toml` :

```
[versions]
# ...
kotlinxCoroutines = "1.7.3"

[libraries]
# ...
kotlinx-coroutines-android = {
```

```
group = "org.jetbrains.kotlinx",
name = "kotlinx-coroutines-android",
version.ref = "kotlinxCoroutines"
}
```

Dans `build.gradle.kts` (module) :

```
dependencies {
    // ...
    implementation(libs.kotlinx.coroutines.android)
}
```

Après toute modification du `build.gradle.kts`, pensez à synchroniser le projet (*Sync Now* dans la barre de notification en haut d'Android Studio).

1.3. Structure des packages

Organisez votre projet en packages dès le départ :

```
com.votregroupe.minigames/
|-- ui/
|   |-- home/
|   |   '-- HomeScreen.kt
|   '-- reaction/
|       '-- ReactionScreen.kt
'-- MainActivity.kt
```

2. Structure attendue

Voici les composables attendus et leurs signatures :

```
// MainActivity.kt
@Composable
fun MiniGamesApp() { ... }

// ui/home/HomeScreen.kt
@Composable
fun HomeScreen(onPlayClick: () -> Unit) { ... }

// ui/reaction/ReactionScreen.kt
@Composable
fun ReactionScreen(onBackClick: () -> Unit) { ... }
```

La fonction `MiniGamesApp` est le point d'entrée de l'UI. Elle décide quel écran afficher en fonction de l'état courant de l'application.

3. Ordre d'implémentation

Pour avancer progressivement, suivez cet ordre :

1. **HomeScreen** : commencez par l'écran le plus simple. Affichez le titre et un bouton "Jouer" qui appelle `onPlayClick`. Ne vous préoccupez pas encore de la navigation.
2. **MiniGamesApp** : mettez en place la navigation avec un `if/else` sur un état booléen. Branchez `HomeScreen` et placez un composable vide pour `ReactionScreen` en attendant.
3. **ReactionScreen – affichage statique** : construisez l'interface sans logique pour l'instant. Affichez la valeur cible, le timer à zéro et les boutons. Vérifiez que tout s'affiche correctement.

4. **ReactionScreen – logique du timer** : ajoutez les etats necessaires et implementez le timer avec `LaunchedEffect`. Le timer doit demarrer, s'incrémenter et s'arreter correctement.
5. **ReactionScreen – phase de resultat** : affichez l'ecart, le message de feedback et gerez le retour a l'accueil.
6. **Aleatoire** : generez aleatoirement la valeur de depart, la vitesse et le sens du timer a chaque nouvelle partie.
7. **Polish** : soignez l'interface – couleurs, espacements, lisibilite du timer.

4. Fonctionnalites attendues

4.1. Ecran d'accueil

L'ecran d'accueil est le point d'entree de l'application. Il doit afficher le nom de l'application ainsi qu'un bouton permettant de lancer le jeu de reaction.

Dans Jetpack Compose, l'interface est *reactive* : elle se recompose automatiquement lorsque l'etat change. Pour conserver une valeur entre deux recompositions, on utilise `remember`. Pour rendre cette valeur observable, on utilise `mutableStateOf`.

```
// La valeur est conservee entre recompositions
var isPlaying by remember { mutableStateOf(false) }

// Modifier l'etat declenche une recomposition
Button(onClick = { isPlaying = true }) {
    Text("Jouer")
}
```

Documentation : developer.android.com/jetpack/compose/state

La navigation entre l'ecran d'accueil et le jeu sera geree avec un simple `if/else` sur un etat boolean dans `MiniGamesApp`.

4.2. Jeu de reaction

Le principe du jeu est simple : un timer defile a l'ecran et le joueur doit appuyer sur un bouton au plus pres d'une **valeur cible** affichee a l'ecran.

Chaque partie est generee aleatoirement selon trois parametres :

- **Valeur de depart** : le timer ne commence pas forcement a zero (ex. : 15 478 ms).
- **Vitesse** : le timer peut s'ecouler plus vite ou plus lentement que la realite.
- **Sens** : le timer peut s'incrémenter ou se decrementer.

Une *coroutine* est une tache asynchrone legere. Dans Compose, **LaunchedEffect** permet de lancer une coroutine liee au cycle de vie d'un composable : elle demarre quand le composable entre en composition et est annulee quand il en sort.

```
// Se relance a chaque fois que "isRunning" change
LaunchedEffect(isRunning) {
    while (isRunning) {
        delay(10L) // attend 10 ms (fonction suspend)
        elapsedMs += step // step peut etre positif ou negatif
    }
}
```

Documentation : [developer.android.com – LaunchedEffect](https://developer.android.com/jetpack/compose/launchedeffect)

Le jeu se deroule en deux phases :

1. **Phase de jeu** : le timer demarre apres appui sur “Demarrer” et defile selon les parametres de la partie. Le joueur appuie sur “Stop!” quand il estime etre proche de la cible.
2. **Phase de resultat** : l'ecart entre la valeur atteinte et la valeur cible est affiche, accompagne d'un message de feedback. Le joueur peut rejouer ou revenir a l'accueil.

Bonus

Ces fonctionnalites ne sont pas obligatoires mais seront valorisees :

- **Timer aveugle** : le timer se masque automatiquement lorsqu'il s'approche de la valeur cible. Le joueur doit alors se fier a son instinct.
- **Vitesse variable** : la vitesse du timer change aleatoirement en cours de partie, rendant l'estimation encore plus difficile.